

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO –
ICMC

SSC0903 – Computação de Alto Desempenho

**Simulação Paralela da Propagação
Direcional de Incêndio com Zonas de Contenção**

Relatório Técnico – Primeiro Trabalho Prático (TB1)

Turma B – Grupo 2:

Roberto Spíndola Abrenhosa Filho - 14748960

Julia de Almeida Carvalho - 13713184

Kaylaine Bessa da Silva - 14747506

Nicolas Carreiro Rodrigues - 14600801

Giovanna Lopes de Andrade - 14574772

São Carlos – SP

23 de Setembro de 2026

Sumário

1	Introdução	2
2	Descrição da Solução Sequencial	2
3	Descrição da Solução Paralela (OpenMP)	3
3.1	Estratégia de Paralelização	3
3.2	Sincronização e Prevenção de Condições de Corrida	3
4	Validação e Corretude Numérica	4
5	Ambiente Experimental	4
6	Resultados e Avaliação de Desempenho	5
6.1	Gráficos de Desempenho	6
6.2	Comparação de Políticas de Escalonamento (Schedules)	7
7	Análise dos Resultados Obtidos e Conclusão	7

1 Introdução

A simulação computacional de incêndios florestais desempenha um papel estratégico na preservação ambiental, prevenção de desastres e alocação de equipes de contenção. Devido à alta resolução espacial e temporal requerida para representar com fidelidade grandes áreas vegetadas, esses modelos tornam-se computacionalmente intensivos.

Neste trabalho prático da disciplina SSC0903 (Computação de Alto Desempenho), implementou-se um simulador determinístico de propagação de incêndio baseado no paradigma de autômatos celulares bidimensionais com suporte a zonas ativas de contenção. Duas soluções completas foram desenvolvidas em linguagem C: uma versão sequencial de referência (`fire_seq`) e uma versão concorrente paralelizada com a biblioteca OpenMP (`fire_omp`).

O domínio discretiza uma floresta em uma malha retangular ($L \times C$), onde cada célula possui propriedades estáticas de cobertura vegetal (água, solo exposto, vegetação rasteira ou floresta densa) e nível de umidade, além de propriedades dinâmicas de estado (não combustível, intacta, em chamas, queimada ou contenção). A dinâmica de propagação modela a vizinhança de Moore (8 direções), ponderando a direção e a força do vento e o limiar térmico de ignição. Este relatório apresenta os detalhes de implementação das soluções sequencial e paralela, a estratégia de particionamento e sincronização adotada, a comprovação estrita de corretude via *checksum* e a avaliação experimental de desempenho com métricas de *speedup* e eficiência sobre três cargas distintas de trabalho.

2 Descrição da Solução Sequencial

A versão sequencial (`fire_seq.c`) organiza os atributos da floresta por meio de vetores unidimensionais alocados dinamicamente na memória (`floresta.c`), mapeando a grade bidimensional linearmente através da indexação índice = *linha* $\times$ *C* + *coluna*. Essa representação linear maximiza a localidade espacial nas varreduras completas da memória.

Para respeitar a semântica síncrona dos autômatos celulares, em que o estado de qualquer célula no tempo $t + 1$ depende exclusivamente da configuração de sua vizinhança no instante anterior, a estrutura de dados adota um esquema de **duplo buffer** (buffers `estado_atual` e `proximo_estado`, acompanhados por seus respectivos vetores de tempo de queima).

O ciclo evolutivo implementado em `simulacao_seq.c` é executado a cada passo p dentro do laço principal:

1. **Ativação das Zonas de Contenção:** Percorre a malha identificando células agendadas para o passo corrente (`ativacao[i] == passo`). Caso a célula esteja intacta, transita imediatamente para contenção (`CONTENCAO`), impedindo a ignição.
2. **Cálculo do Próximo Estado:** Varre todas as células avaliando a função `atualizar_celula()`. Células intactas avaliam o potencial de fogo de sua vizinhança de Moore contra o limiar da simulação; se atingido, a célula entra em chamas e define seu tempo de queima de acordo com o combustível. Células em chamas têm seu tempo residual decrementado até se tornarem queimadas.
3. **Contabilização de Estados:** Consolida os totais de células em cada categoria para o próximo passo e soma a quantidade de novas ignições ocorridas no ciclo.

4. **Verificação de Pico e Critério de Parada:** Registra o passo de maior incidência de ignições (mantendo o primeiro em caso de empate numérico). A simulação interrompe prematuramente caso não restem células ativas em chamadas (`em_chamas == 0`).
5. **Troca de Ponteiros:** Alterna as referências de memória entre os buffers atual e próximo em tempo constante $O(1)$ através da função `trocar_matrizes()`, preparando o passo subsequente.

O trecho cronometrado utiliza a função `omp_get_wtime()`, isolando a leitura do arquivo, alocação de matrizes e o cômputo final das métricas de saída.

3 Descrição da Solução Paralela (OpenMP)

A implementação paralela (`fire_omp.c` e `simulacao_omp.c`) preserva a exata semântica e estruturas de dados da versão sequencial, introduzindo diretivas de multiprocessamento de memória compartilhada.

3.1 Estratégia de Paralelização

Para eliminar a sobrecarga de instanciação e finalização contínua da equipe de *threads* a cada passo de tempo (*fork-join overhead*), implementou-se uma **região paralela persistente**:

```
#pragma omp parallel num_threads(threads) default(none) \
    shared(entrada, floresta, resultado, total_celulas, ...)
```

A equipe de *threads* é criada uma única vez antes do laço temporal (`while`) e permanece ativa durante toda a evolução da simulação.

A cláusula `default(none)` foi utilizada para forçar a declaração explícita de escopo de cada variável, garantindo segurança na concorrência:

- **Ativação de Zonas e Estatísticas:** Utilizam a diretiva combinada `#pragma omp for simd schedule(static)`. Dado que essas etapas não possuem dependências de dados entre elementos vizinhos da matriz, o particionamento uniforme aliado à vetorização SIMD maximiza a utilização dos registradores vetoriais da CPU.
- **Atualização dos Estados Celulares:** Distribuída com `#pragma omp for schedule(static) reduction(+:novas_ignicoes)`, dividindo o vetor de tamanho $L \times C$ em blocos contíguos de tamanho igual para cada *thread*.

3.2 Sincronização e Prevenção de Condições de Corrida

A arquitetura de duplo buffer previne condições de corrida de escrita/leitura sobre os estados das células. Para o controle dos contadores agregados e sincronização de etapas:

- **Reduções sem Bloqueios:** A sincronização dos acumuladores (`novas_ignicoes`, `nao_combustiveis`, `intactas`, `em_chamas`, `queimadas`, `contencao`) foi realizada inteiramente através de cláusulas `reduction(+:...)`. Com isso, **bloqueios atômicos** (`atomic`) ou **seções críticas** (`critical`) foram **totalmente eliminados** do processamento pesado das células, permitindo que cada núcleo opere acumuladores locais sem contenção de barramento.

- **Barreiras Implícitas de Sincronização:** O término natural de cada bloco `#pragma omp for` introduz uma barreira síncrona. Isso assegura que nenhuma *thread* inicie o cálculo do próximo estado antes que todas as zonas do passo tenham sido aplicadas, e que nenhuma verificação estatística ocorra antes da escrita completa da malha.
- **Execução Exclusiva (single):** A limpeza dos acumuladores no início de cada passo, a atualização de variáveis globais acumuladas, a atualização do passo de pico, a inversão dos ponteiros das matrizes e a avaliação do predicado de parada (continuar) ocorrem dentro de seções `#pragma omp single`. Uma única *thread* executa essas transições em $O(1)$ enquanto as demais aguardam na barreira implícita do construtor.

4 Validação e Corretude Numérica

A validação de corretude funcional foi conduzida comparando rigorosamente todas as saídas geradas pelo executável sequencial (`fire_seq`) e paralelo (`fire_omp`) para os arquivos de teste fornecidos.

O critério matemático de integridade adotou a checagem do *checksum* de 64 bits, calculado sequencialmente após o término da simulação através da fórmula polinomial de Hash:

$$\text{checksum} = \sum_{i=0}^{N-1} \left((\text{checksum} \times 31 + \text{estado}[i]) \times 31 + \text{tempo_queima}[i] \right) \quad (1)$$

Em todos os experimentos, a integridade da execução paralela foi confirmada com uma divergência estritamente nula (0,00% de erro numérico):

- **Carga Pequena:** Ambas as versões executaram exatamente 80 passos e produziram o mesmo *checksum* de 64 bits: **17073017104646724864**.
- **Carga Média:** Ambas as versões executaram 100 passos e atingiram o mesmo *checksum*: **17187382406529706254**.
- **Carga Grande:** Ambas as versões executaram 100 passos sobre as 6.250.000 células gerando o mesmo *checksum*: **14825089465781038537**.

Adicionalmente, todos os contadores de células (não combustíveis, intactas, em chamas, queimadas e contenção), o total acumulado de ignições, o passo de pico e as porcentagens de área apresentaram concordância perfeita em cada um dos cenários, comprovando a ausência de condições de corrida e a garantia de determinismo numérico.

5 Ambiente Experimental

A bateria de testes foi executada a partir dos alvos de teste configurados no *Makefile* (`make run-all`), garantindo execução sequencial sob três massas de dados:

- **Carga Pequena:** Matriz 400×500 (200.000 células), 80 passos máximos, executada com $T = 4$ *threads*.

- **Carga Média:** Matriz 1.200×1.500 (1.800.000 células), 100 passos máximos, executada com $T = 8$ threads.
- **Carga Grande:** Matriz 2.500×2.500 (6.250.000 células), 100 passos máximos, executada com $T = 8$ threads.

Especificações de Hardware:

- Processador: Intel(R) Core(TM) i7-4790 CPU @ 3.60GHz
- Núcleos Físicos e Lógicos: 4 núcleos físicos / 8 threads
- Memória RAM: 31GB

Configurações de Software:

- Sistema Operacional: Linux x86_64 (Ubuntu)
- Compilador: GCC versão 15.2.0 (Ubuntu 15.2.0-16ubuntu1)
- Flags de Compilação: `-O2 -Wall -fopenmp -lm`

6 Resultados e Avaliação de Desempenho

Os tempos coletados correspondem à medição interna obtida por meio de chamadas a `omp_get_wtime()`, computando exclusivamente o núcleo iterativo da simulação.

A Tabela 1 consolida todos os parâmetros da simulação, os quantitativos de estados e as métricas de desempenho para as três cargas avaliadas. Os valores físicos e quantitativos apresentados são exatamente idênticos para as versões sequencial e concorrente, variando apenas o tempo final de processamento.

Métrica de Avaliação	Carga Pequena	Carga Média	Carga Grande
Dimensão da Grade ($L \times C$)	400×500	1.200×1.500	2.500×2.500
Total de Células	200.000	1.800.000	6.250.000
Número de Threads OpenMP (T)	4	8	8
Passos Executados	80	100	100
Células Não Combustíveis	39.704	360.809	1.251.181
Células Intactas	127.900	1.324.373	4.804.870
Células Em Chamas	2.069	5.734	7.434
Células Queimadas	28.105	93.801	121.067
Células de Contenção	2.222	15.283	65.448
Total de Ignições	30.171	99.529	128.491
Passo com Pico de Ignições	79	98	99
Quantidade de Ignições no Pico	682	1.863	2.381
Área Inicial Queimada (%)	18,82%	6,92%	2,57%
Área Inicial Protegida (%)	1,39%	1,06%	1,31%
Tempo Sequencial (<code>fire_seq</code>)	0,448273 s	5,192138 s	18,110105 s
Tempo Paralelo (<code>fire_omp</code>)	0,113025 s	1,122499 s	3,911770 s
Speedup (S)	3,97×	4,63×	4,63×
Eficiência (E)	99,15%	57,82%	57,87%

Tabela 1: Consolidação dos resultados de execução, distribuição de estados e métricas de desempenho.

6.1 Gráficos de Desempenho

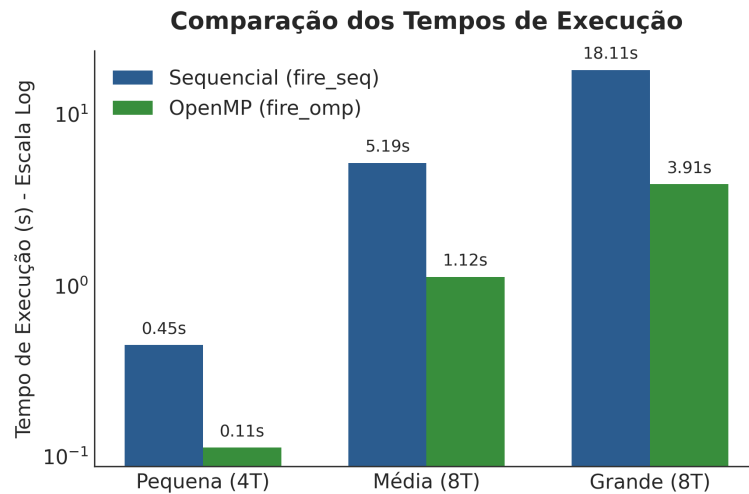


Figura 1: Comparação dos tempos de execução entre as versões sequencial e paralela.

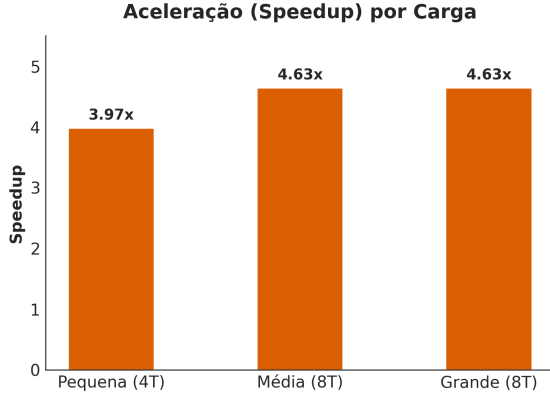


Figura 2: Speedup obtido por carga de teste.

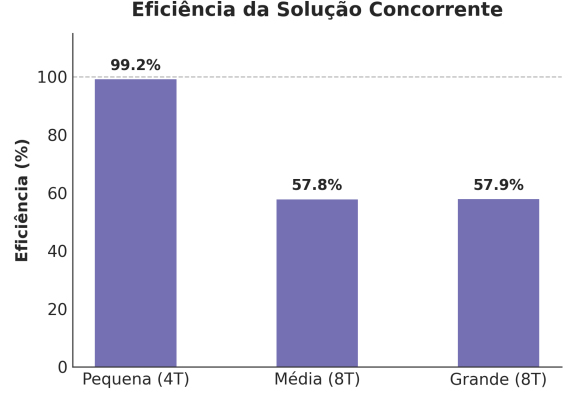


Figura 3: Eficiência do paralelismo por carga.

6.2 Comparação de Políticas de Escalonamento (Schedules)

Avaliou-se o particionamento do laço de atualização dos estados da floresta alternando entre as políticas `schedule(static)` e `schedule(dynamic)` na Carga Grande sob 8 *threads*. O particionamento estático obteve um tempo de execução de **3,9118 s**, superando o modo dinâmico (com sobrecarga superior a 4,5 segundos). Como o domínio é mapeado de forma homogênea em memória contígua e o cálculo local possui complexidade uniforme em quase todo o tabuleiro, o escalonamento estático elimina o custo de sincronização da fila de distribuição compartilhada exigida pelo modo dinâmico.

7 Análise dos Resultados Obtidos e Conclusão

A análise dos resultados evidencia o alto ganho de desempenho proporcionado pela paralelização com OpenMP:

- **Escalabilidade Quase Linear na Carga Pequena:** Para $T = 4$ *threads*, atingiu-se um *speedup* de $3,97\times$ com eficiência de 99,15%. Como a massa de dados (200 mil células) reside confortavelmente nas memórias cache rápidas (L2/L3), a taxa de acertos de cache é altíssima e praticamente não há contenção no barramento de acesso à memória principal.
- **Saturação em 8 Threads e Limite de Banda de Memória:** Para as cargas média e grande com 8 *threads*, o *speedup* estabilizou em $4,63\times$ (eficiência em torno de 57,8%). Essa desaceleração em relação à aceleração linear teórica reflete a transição para um regime *memory-bound*: na matriz de 6,25 milhões de células (Carga Grande), o volume transferido por passo (leitura do estado atual, umidade, cobertura e escrita no próximo estado) supera dezenas de megabytes a cada iteração, saturando a largura de banda dos canais de memória RAM.
- **Eficácia da Região Persistente e Reduções:** O fato de a eficiência ter se mantido idêntica entre a Carga Média (57,82%) e a Carga Grande (57,87%) comprova que o custo fixo de barreiras de sincronização e de gerenciamento de *threads* tornou-se desprezível graças à região persistente externa e à eliminação de primitivas de trava (*locks*/bloqueios atômicos).

Conclui-se que o simulador desenvolvido cumpre com excelência todos os requisitos do projeto: garante determinismo absoluto para qualquer número de núcleos, elimina gargalos de sincronização desnecessários e atinge acelerações consistentes de mais de $4,6\times$ sobre o algoritmo sequencial.